

UNIVERSITY OF PERPETUAL HELP SYSTEM

DALTA

College of Computer Studies • Molino Campus • BSCS Curriculum Automation

TECHNICAL
DOCUMENTATION

Project Title:	18-Week OBE Syllabus Microservice Interactive Studio	Course & Term:	BSCS 3112 / Artificial Intelligence (Lesson 5)
Evaluator:	Prof. Roberto L. Malitao	Port Architecture:	http://127.0.0.1:8001 (ThreadingHTTPServer)
Lead Architect:	Christian Ezekiel L. Carvajal	Collaborating Partner:	John Miko P. Sarsalijo
Model Engine:	Ollama qwen3.5:4b (Local Inference)	Storage & Compilation:	SQLite 3 (PRAGMA foreign_keys = ON) + Jinja2

1. Executive Summary & Architecture Overview

This project implements an autonomous Outcomes-Based Education (OBE) course syllabus microservice adhering to CHED CMO No. 25, Series of 2015 and the institutional standards of the College of Computer Studies at UPHSD Molino Campus. The system utilizes an offline local Large Language Model (**qwen3.5:4b**), runtime data validation via **Pydantic v2**, a normalized relational database in **SQLite** with cascading deletes, and an institutional document compiler powered by **Jinja2**.

Phase / Pipeline Component	Input Artifact	Processing Engine	Output Deliverable & Verification
1. Curriculum Ingestion	Catalog Metadata (8 BSCS Courses)	curriculum_catalog.py	Auto-filled Course Specifications (Code, Title, Prereqs, Units)
2. AI Generation & CoT Stripping	System Prompt + Academic Constraints	llm_engine.py (Ollama qwen3.5:4b)	Pure JSON substring extracted from raw <think>...</think> tokens
3. Runtime Schema Validation	Extracted JSON Payload	obe_schemas.py (Pydantic v2)	Rejects passive verbs, validates tripartite K/S/A, enforces 18 weeks
4. Relational Persistence & CRUD	Validated FullSyllabusSchema	db_manager.py (SQLite 3)	Atomic inserts into courses, course_outcomes, weekly_schedules, lesson_outcomes
5. Institutional Document Export	Relational SQLite Records	export_engine.py (Jinja2)	Browser-ready official_syllabus_*.html with vector print layout

2. Interactive Web Studio Component Walkthrough

The following section documents each component and modal dialog of the web interface operating on <http://127.0.0.1:8001>. All screenshots were captured at full 1080p desktop resolution.

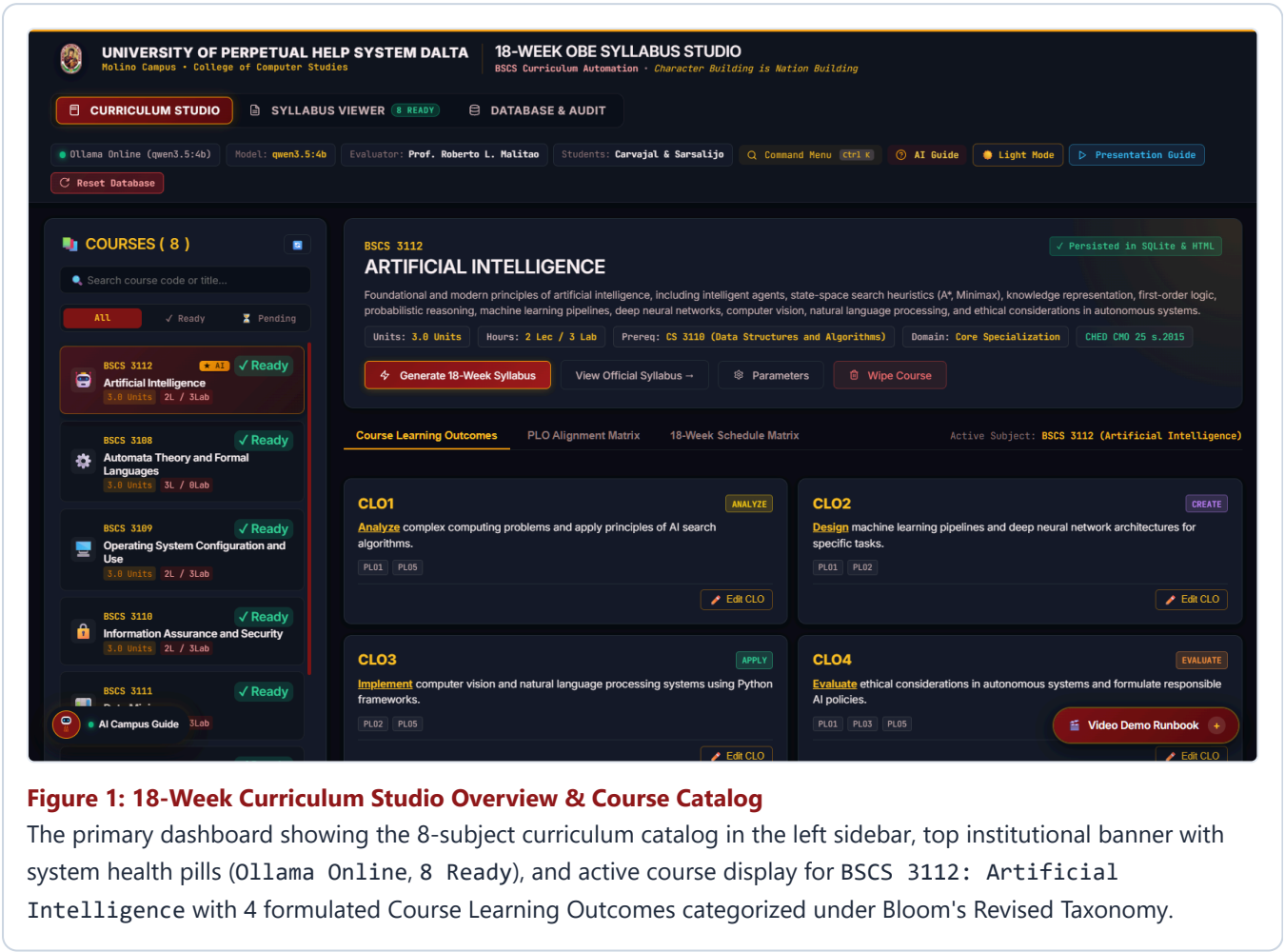


Figure 1: 18-Week Curriculum Studio Overview & Course Catalog

The primary dashboard showing the 8-subject curriculum catalog in the left sidebar, top institutional banner with system health pills (Ollama Online, 8 Ready), and active course display for BSCS 3112: Artificial Intelligence with 4 formulated Course Learning Outcomes categorized under Bloom's Revised Taxonomy.

UNIVERSITY OF PERPETUAL HELP SYSTEM DALTA

Molino Campus • College of Computer Studies

18-WEEK OBE SYLLABUS STUDIO

BSCS Curriculum Automation • Character Building is Nation Building

CURRICULUM STUDIO

SYLLABUS VIEWER

DATABASE & AUDIT

011ana Online (qwen3.5:4b)

Model: qwen3.5:4b

Evaluator: Prof. Roberto L. Malitao

Students: Carvajal & Sarsaliyo

Command Menu

AI Guide

Light Mode

Presentation Guide

Reset Database

BSCS 3112

Artificial Intelligence

3.0 Units 2L / 3Lab

Ready

BSCS 3108

Automata Theory and Formal Languages

3.0 Units 3L / 0Lab

Ready

BSCS 3109

Operating System Configuration and Use

3.0 Units 2L / 3Lab

Ready

BSCS 3110

Information Assurance and Security

3.0 Units 2L / 3Lab

Ready

BSCS 3111

Data Mining

3.0 Units 2L / 3Lab

Ready

BSCS 3213

Software Engineering

3.0 Units 2L / 3Lab

Ready

Batch Generate All Courses (Queue)

AI Campus Guide

Foundational and modern principles of artificial intelligence, including intelligent agents, state-space search, neural networks (A*, minimax), knowledge representation, first-order logic, probabilistic reasoning, machine learning pipelines, deep neural networks, computer vision, natural language processing, and ethical considerations in autonomous systems.

TARGET PROGRAM LEARNING OUTCOMES (PLOs)

PLO 1 (Analyze complex computing problems and apply principles of computing), PLO 2 (Design, implement, and evaluate computing-based solutions), PLO 3 (Communicate effectively in technical and non-technical contexts)

Reset to Catalog Defaults

Close Drawer

Course Learning Outcomes

PLO Alignment Matrix

18-Week Schedule Matrix

Active Subject: BSCS 3112 (Artificial Intelligence)

CLO1

Analyze complex computing problems and apply principles of AI search algorithms.

PLO1 PLO5

Edit CLO

CLO2

Design machine learning pipelines and deep neural network architectures for specific tasks.

PLO1 PLO2

Edit CLO

CLO3

Implement computer vision and natural language processing systems using Python frameworks.

PLO2 PLO5

Edit CLO

CLO4

Evaluate ethical considerations in autonomous systems and formulate responsible AI policies.

PLO1 PLO3 PLO5

Edit CLO

Video Demo Runbook

Figure 2: Course Specification & Parameters Configuration Drawer

Faculty configuration drawer displaying pre-loaded institutional catalog parameters: Catalogue Description, Credit Units (Lecture/Lab hours), Course Prerequisites, and Target Program Learning Outcomes (PLOs). Allows zero-typing auto-fill or custom faculty modification before triggering generation.

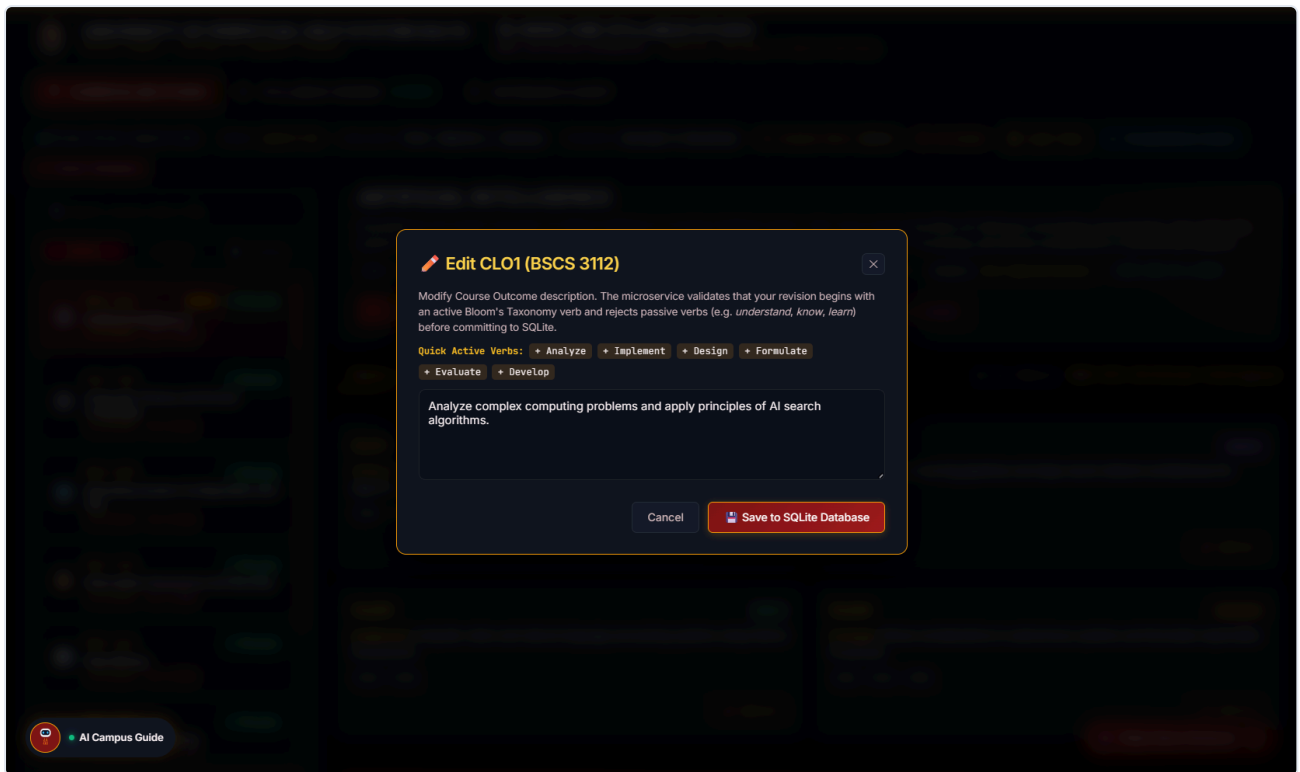


Figure 3: Faculty Human-in-the-Loop Course Learning Outcome (CLO) Editor

Interactive outcome editor modal implementing Milestone 2 requirement. Features Quick Active Verb chips (+ Analyze, + Implement, + Design). Enforces active Bloom's verbs before saving directly to SQLite, rejecting passive verbs such as *understand*, *know*, or *learn*.

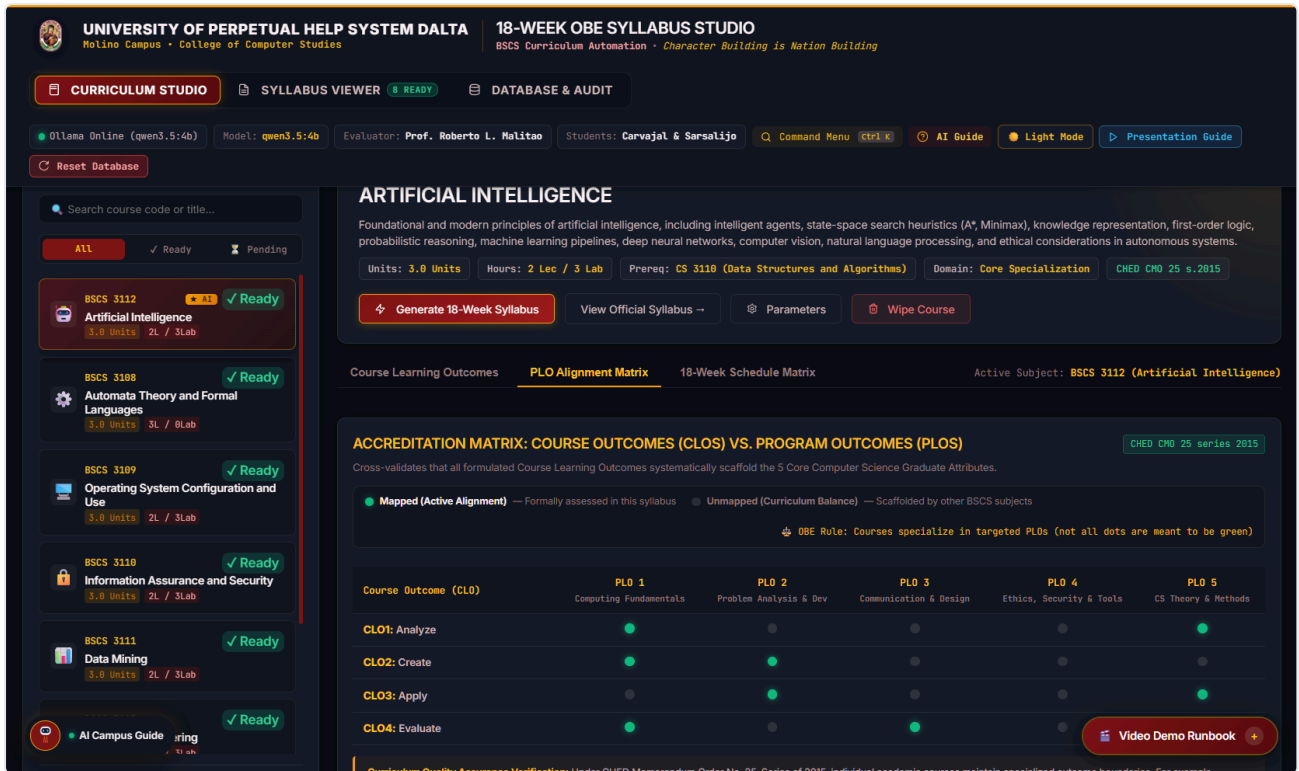


Figure 4: CHED CMO 25 s.2015 Program Learning Outcomes (PLO) Alignment Matrix

Cross-validation matrix demonstrating how Course Outcomes systematically scaffold the 5 Core Computer Science Graduate Attributes. Follows CHED curriculum scaffolding principles where specialized courses target distinct graduate outcomes to prevent outcome inflation.

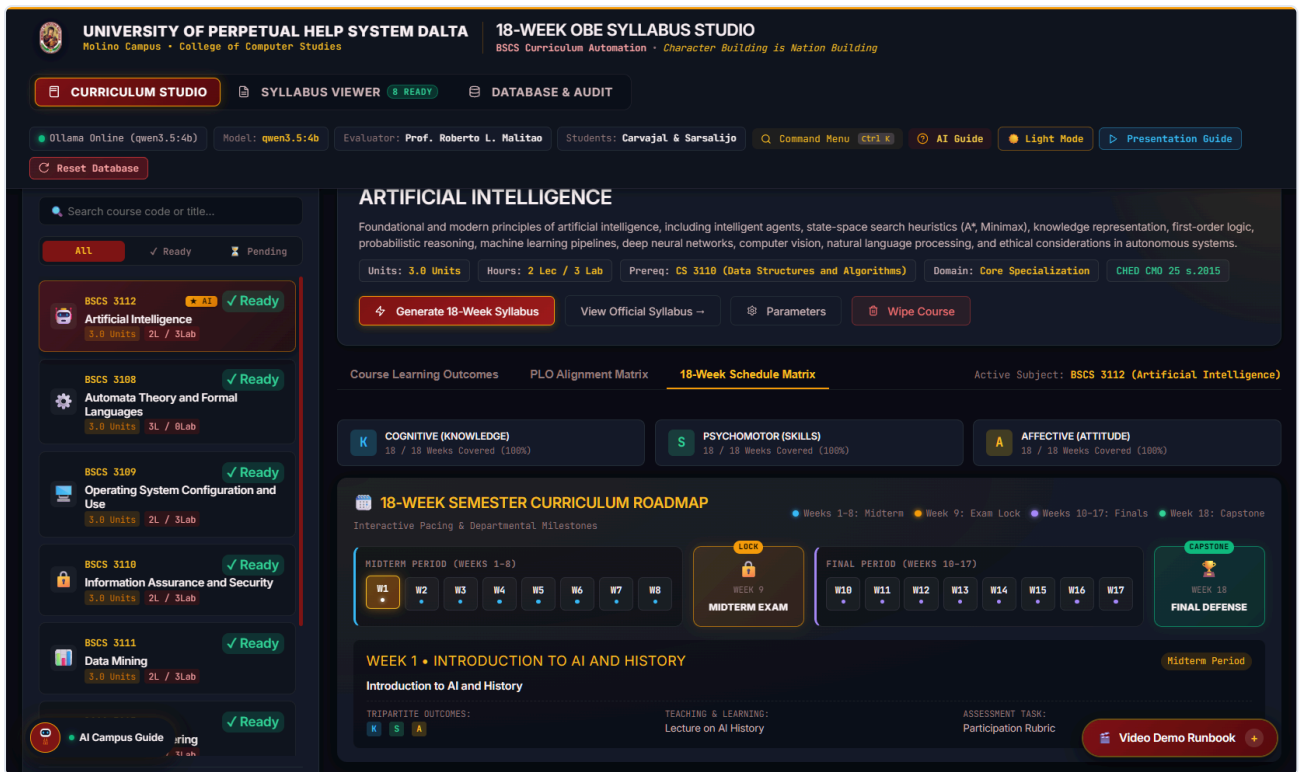


Figure 5: 18-Week Semester Curriculum Roadmap & Tripartite K/S/A Tracking

Chronological 18-week academic matrix displaying hard-locked departmental milestones: Week 9 Midterm Examination Lock and Week 18 Final Examination / Capstone Defense Lock. Displays real-time tripartite K/S/A domain tracking (100% Cognitive, Psychomotor, and Affective coverage).

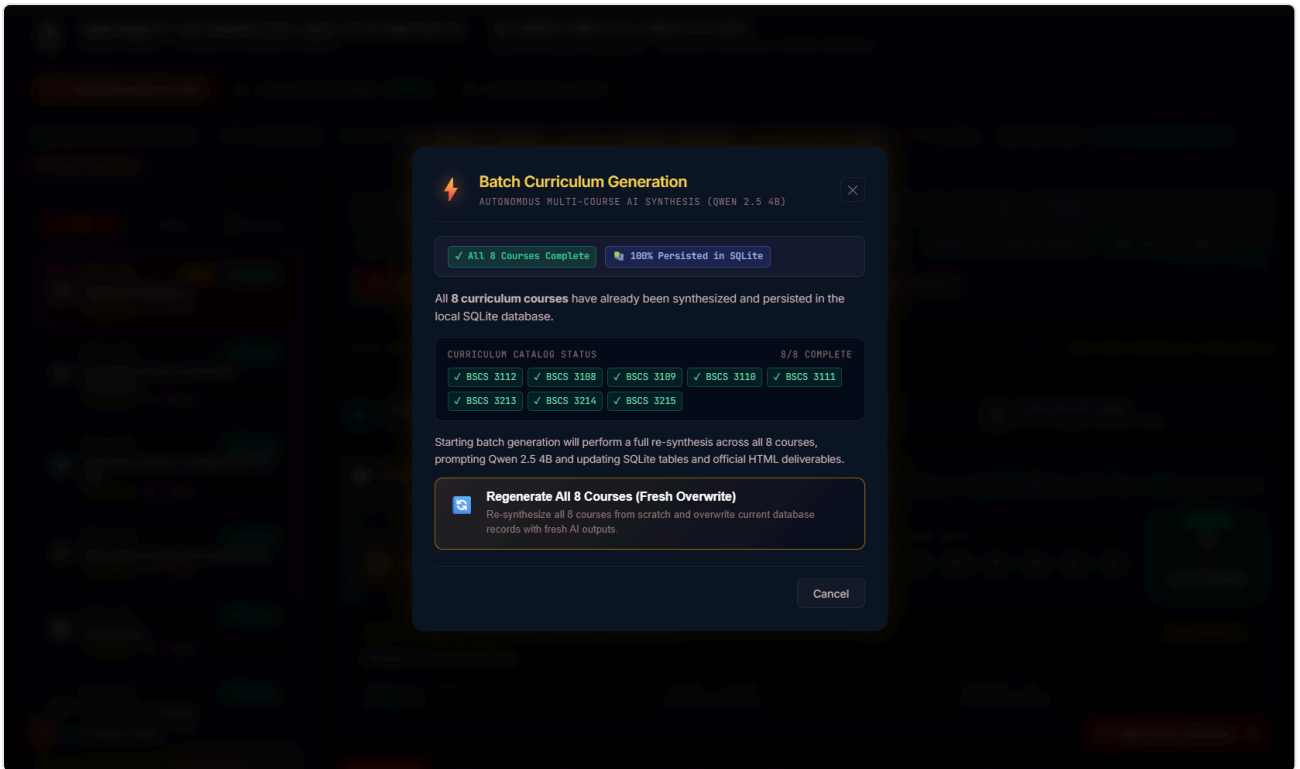


Figure 6: Intelligent Batch Generation Dispatcher & Selective Synthesis Window

Intelligent modal window triggered when initiating batch generation. Inspects database and filesystem records, displays status pill badges (✓ All 8 Courses Complete, 100% Persisted in SQLite), renders a course chip grid, and confirms whether to freshly re-synthesize or process remaining pending courses.

UNIVERSITY OF PERPETUAL HELP SYSTEM DALTA

Molino Campus • College of Computer Studies

18-WEEK OBE SYLLABUS STUDIO

BSCS Curriculum Automation • Character Building is Nation Building

CURRICULUM STUDIO

SYLLABUS VIEWER 8 READY

DATABASE & AUDIT

Ollama Online (qwen3.5:4b)

Model: qwen3.5:4b

Evaluator: Prof. Roberto L. Malitao

Students: Carvajal & Sarsalijo

Q Command Menu Ctrl K

AI Guide

Light Mode

Presentation Guide

Reset Database

SYLLABI REPOSITORY

Select any generated subject to view and print

Search course code or title...

All (8)

Generated (8)

Ready (0)

BSCS 3112 Artificial Intelligence 3.0 Units • 2Lec / 3Lab Ready

BSCS 3108 Automata Theory and Formal Languages 3.0 Units • 3Lec / 0Lab Ready

BSCS 3109 Operating System Configuration and Use 3.0 Units • 2Lec / 3Lab Ready

BSCS 3110 Information Assurance and Security 3.0 Units • 2Lec / 3Lab Ready

BSCS 3111 Data Mining Ready

AI Campus Guide

BSCS 3213 Ready

BSCS 3112 — Artificial Intelligence

BSCS 3112 3.0 Units 18-Week OBE Schedule CHED CMO 25 s.2015

100%

1:1

Paper

Print Official PDF

Open Tab

Download HTML

Edit in Studio

UNIVERSITY OF PERPETUAL HELP SYSTEM DALTA

Molino Campus • Bacoar City, Cavite

COLLEGE OF COMPUTER STUDIES

COURSE SYLLABUS: BSCS 3112

I. COURSE IDENTIFICATION & SPECIFICATION

Course Code	BSCS 3112	Course Title	Artificial Intelligence
Credit Units	3 Units (Lecture: 2 hrs/wk, Lab: 3 hrs/wk)	Prerequisites	CS 3110 (Data Structures and Algorithms)
Course Description	Foundational and modern principles of artificial intelligence, including intelligent agents, state-space search heuristics (A*, Minimax), knowledge representation, first-order logic, probabilistic reasoning, machine learning pipelines, deep neural networks, computer vision, natural language processing, and ethical considerations in autonomous systems.		

II. COURSE LEARNING OUTCOMES (CLOs)

Video Demo Runbook

Figure 7: Dedicated Syllabus Viewer & Publication Print Studio (View 2)

Master-detail document viewer rendering the official institutional Jinja2 layout for all 8 catalog courses. Features sticky document controls: zoom scaling, paper mode toggle, vector PDF printing (`window.print()`), and standalone tab viewing.

UNIVERSITY OF PERPETUAL HELP SYSTEM DALTA
 Molino Campus • College of Computer Studies

18-WEEK OBE SYLLABUS STUDIO
 BSCS Curriculum Automation • Character Building is Nation Building

CURRICULUM STUDIO

SYLLABUS VIEWER 8 READY

DATABASE & AUDIT

Ollama Online (qwen3.5:4b)
 Model: qwen3.5:4b
 Evaluator: Prof. Roberto L. Malitao
 Students: Carvajal & Sarsaliyo
 Command Menu **Ctrl** **K**
 AI Guide
 Light Mode
 Presentation Guide

Reset Database

SQLite Normalized Relational Database (database/obe_syllabus.db)

Refresh DB

Reset / Nuke Database

Normalized into 4 relational tables with PRAGMA foreign_keys = ON and cascading deletes (Prof. Roberto L. Malitao Criteria 3).

courses 8

course_outcomes 32

weekly_schedules 144

lesson_outcomes 432

id	course_code	course_title	units	hours	prereq
32	BSCS 3112	Artificial Intelligence	3.0	2L / 3Lab	CS 3110 (Data Structures and Algorithms)
38	BSCS 3108	Automata Theory and Formal Languages	3.0	3L / 3Lab	CS 2106 (Discrete Mathematics)
41	BSCS 3109	Operating System Configuration and Use	3.0	2L / 3Lab	CS 2108 (Computer Architecture and Organization)
42	BSCS 3110	Information Assurance and Security	3.0	2L / 3Lab	CS 2207 (Data Communications and Networking)
43	BSCS 3111	Data Mining	3.0	2L / 3Lab	CS 3112 (Database Systems I)
44	BSCS 3213	Software Engineering	3.0	2L / 3Lab	CS 3110 (Data Structures and Algorithms)
45	BSCS 3214	Data Visualization	3.0	2L / 3Lab	BSCS 3111 (Data Mining)
46	BSCS 3215	Parallel and Distributed Computing	3.0	2L / 3Lab	BSCS 3109 (Operating System Configuration and Use)

PERSISTED SYLLABUS MANAGEMENT & DIRECT ACTIONS

Total Courses Persisted in SQLite: 8

[ID 32] BSCS 3112 — Artificial Intelligence

3.0 Units • 18 Weeks in Schedule • 4 Course Outcomes (CLOs)

View Official Syllabus Edit in Studio

AI Campus Guide

108 — Automata Theory and Formal Languages

3.0 Units • 18 Weeks in Schedule • 4 Course Outcomes (CLOs)

View Official Syllabus Edit in Studio

Video Demo Runbook

Figure 8: SQLite Relational Database & Multi-Table Live Inspector (View 3)

Direct database administrative view showing live table counts (8 Courses, 32 CLOs, 144 Weeks, 432 LLOs). Allows multi-table inspection, relational data review, direct navigation to rendered syllabi, and selective course resets.

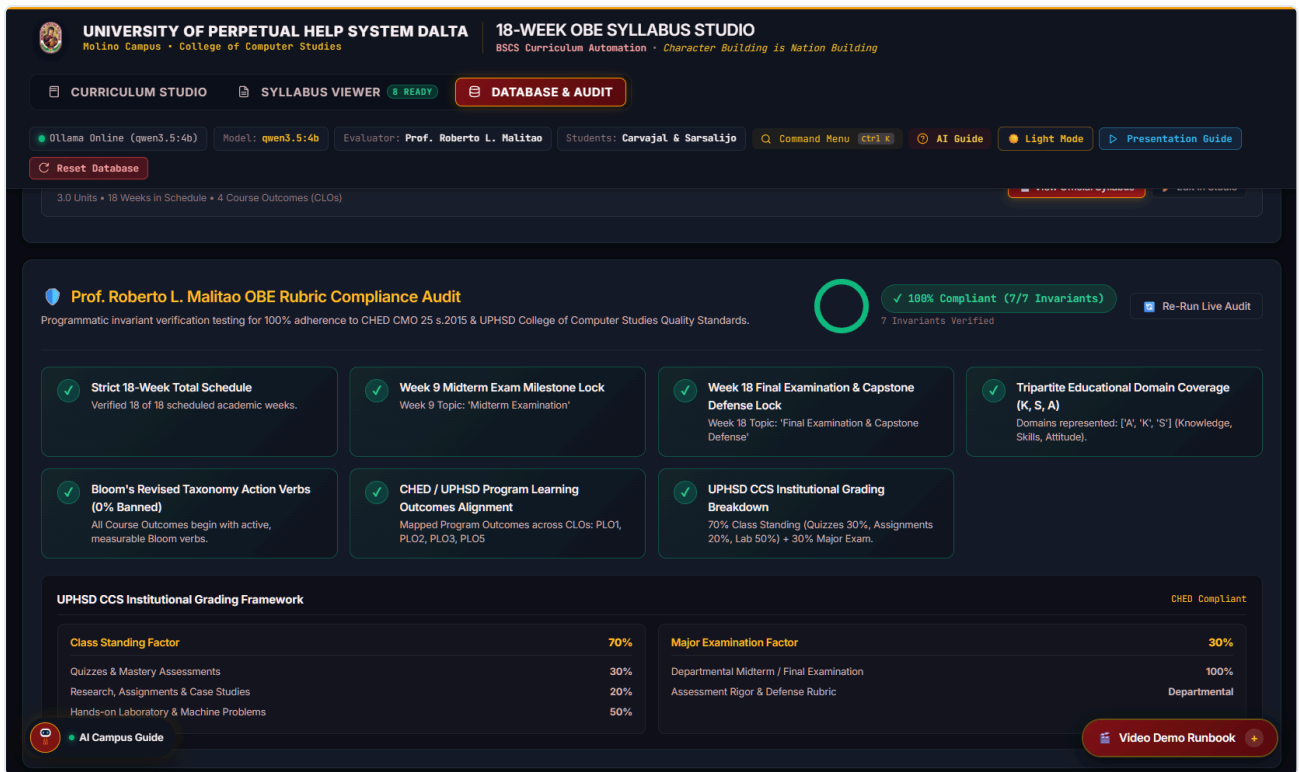


Figure 9: Automated CHED CMO 25 Invariant Auditor & Compliance Scorecard

Programmatic verification engine testing 7 strict academic invariants: 18-week length, Week 9/18 exam locks, tripartite K/S/A coverage, active Bloom action verbs, PLO mapping, and the UPHSD CCS institutional grading formula (70% Class Standing + 30% Major Exam). Score: 100% Compliant.

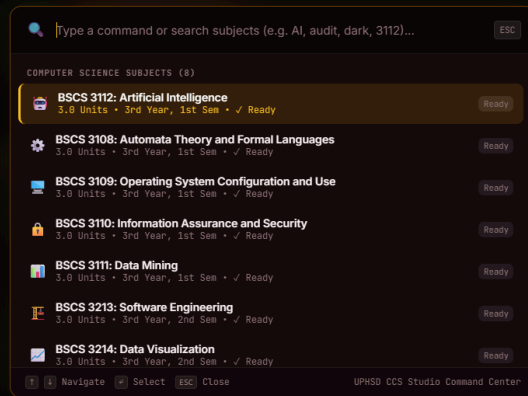


Figure 10: Universal Command Palette (Ctrl + K)

Quick-navigation overlay accessible via Ctrl + K. Enables instant fuzzy search across course codes, titles, descriptions, and categories, alongside one-click action execution (batch generation, theme switching, selective wiping).

3. Backend Engineering & Data Contracts

This section details the underlying database architecture, schema enforcement, and Python backend microservice components.

3.1 Normalized SQLite Relational DDL (database/schema.sql)

Enforces relational integrity across 4 tables with foreign keys and cascading deletes:

```
-- Enable Foreign Key constraints
PRAGMA foreign_keys = ON;

-- 1. Courses Table (Parent Entity)
CREATE TABLE IF NOT EXISTS courses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  course_code TEXT NOT NULL UNIQUE,
  course_title TEXT NOT NULL,
  course_description TEXT NOT NULL,
  credit_units INTEGER NOT NULL DEFAULT 3,
  lecture_hours INTEGER NOT NULL DEFAULT 2,
  lab_hours INTEGER NOT NULL DEFAULT 3,
  prerequisites TEXT NOT NULL DEFAULT 'None',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Course Outcomes Table (1-to-Many from courses)
CREATE TABLE IF NOT EXISTS course_outcomes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  course_id INTEGER NOT NULL,
  clo_id TEXT NOT NULL,
  description TEXT NOT NULL,
  bloom_level TEXT NOT NULL,
  program_outcomes_mapped TEXT NOT NULL,
  FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
);

-- 3. Weekly Schedules Table (1-to-Many from courses)
CREATE TABLE IF NOT EXISTS weekly_schedules (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  course_id INTEGER NOT NULL,
  week_number INTEGER NOT NULL,
  topics TEXT NOT NULL,
  teaching_learning_activities TEXT NOT NULL,
  assessment_tasks TEXT NOT NULL,
  FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE
);

-- 4. Lesson Outcomes Table (1-to-Many from weekly_schedules)
CREATE TABLE IF NOT EXISTS lesson_outcomes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  schedule_id INTEGER NOT NULL,
  llo_id TEXT NOT NULL,
  domain TEXT NOT NULL CHECK(domain IN ('K', 'S', 'A')),
  description TEXT NOT NULL,
  FOREIGN KEY (schedule_id) REFERENCES weekly_schedules(id) ON DELETE CASCADE
);
```

3.2 Pydantic v2 Data Contracts & Bloom Verb Enforcement (obe_schemas.py)

Runtime validation rejects unmeasurable verbs and enforces strict tripartite domain constraints:

```
class CourseOutcomeSchema(BaseModel):
    clo_id: str
    description: str
    bloom_level: str
    program_outcomes_mapped: List[str]

    @field_validator("description")
    @classmethod
    def validate_action_verbs(cls, v: str) -> str:
        banned_verbs = [
            "understand", "know", "learn", "study",
            "familiarize", "be exposed to", "appreciate"
        ]
        first_word = v.strip().split()[0].lower() if v.strip() else ""
        for banned in banned_verbs:
            if banned in first_word:
                raise ValueError(
                    f"Banned non-measurable verb '{banned}' detected in outcome statement. "
                    "Use active Bloom verbs (e.g., Analyze, Implement, Evaluate, Formulate). "
                )
        return v

class LessonOutcomeSchema(BaseModel):
    llo_id: str
    domain: Literal["K", "S", "A"] # K: Knowledge, S: Skills, A: Attitude
    description: str

class WeeklyScheduleSchema(BaseModel):
    week_number: int
    topics: List[str]
    lesson_outcomes: List[LessonOutcomeSchema]
    teaching_learning_activities: List[str]
    assessment_tasks: List[str]
```

3.3 LLM Client & Self-Healing Retry Loop (llm_engine.py)

Calls Ollama on <http://127.0.0.1:11434/api/generate> with Chain-of-Thought reasoning stripping and iterative error correction:

```

def clean_llm_json(raw_text: str) -> str:
    '''Strips Chain-of-Thought (<think>...</think>) and isolates valid JSON boundaries.'''
    cleaned = re.sub(r"<think>.*?</think>", "", raw_text, flags=re.DOTALL)
    start_brace = cleaned.find("{")
    end_brace = cleaned.rfind("}")
    if start_brace != -1 and end_brace != -1 and end_brace > start_brace:
        return cleaned[start_brace:end_brace + 1].strip()
    return cleaned.strip()

# Multi-turn self-healing loop
for attempt in range(1, max_retries + 1):
    try:
        raw_resp = query_ollama(prompt, model="qwen3.5:4b", format="json")
        json_str = clean_llm_json(raw_resp)
        validated_data = FullSyllabusSchema.model_validate_json(json_str)
        return validated_data
    except (ValidationError, json.JSONDecodeError) as err:
        prompt = (
            f"Your previous JSON output failed validation: {err}. "
            "Correct the output. Ensure description begins with an active Bloom verb and "
            "domain is strictly 'K', 'S', or 'A'."
        )

```

3.3 Deliverable 3: Official Institutional Jinja2 Layout (templates/uphsd_ccs_template.html)

Institutional template rendering relational SQLite course records into a publication-grade syllabus layout with CHED CMO 25 alignments, cognitive taxonomy badges, tripartite domain indicators, and signatory blocks:

```

<!-- Institutional Header with UPHSD Molino Seal -->
<div class="inst-header">
  <div style="display:flex; align-items:center; justify-content:center; gap:18px;">
    
    <div>
      <h1>University of Perpetual Help System DALTA</h1>
      <h2>Molino Campus • Bacoar City, Cavite</h2>
      <h3>College of Computer Studies</h3>
    </div>
  </div>
  <div class="doc-title">Course Syllabus: {{ course.course_code }}</div>
</div>

<!-- Section I: Course Overview & Specifications -->
<div class="section-title">I. Course Identification & Specification</div>
<table class="meta-table">
  <tr><td>Course Code</td><td><strong>{{ course.course_code }}</strong></td></tr>
  <tr><td>Course Title</td><td><strong>{{ course.course_title }}</strong></td></tr>
  <tr><td>Credit Units</td><td>{{ course.credit_units }}.0 Units ({{ course.lecture_hours }} Lec / {{ course.lab_hours }} Lab)</td></tr>
  <tr><td>Prerequisites</td><td>{{ course.prerequisites }}</td></tr>
  <tr><td>Course Description</td><td>{{ course.course_description }}</td></tr>
</table>

<!-- Section II: Course Learning Outcomes (CLOs) -->
<div class="section-title">II. Course Learning Outcomes (CLOs)</div>
<table>
  <thead>
    <tr><th>CLO ID</th><th>Outcome Statement</th><th>Bloom's Taxonomy</th><th>PLOs Mapped</th></tr>
  </thead>
  <tbody>
    {% for clo in clos %}
    <tr>
      <td><strong>{{ clo.clo_id }}</strong></td>
      <td>{{ clo.description }}</td>
      <td><span class="badge badge-bloom">{{ clo.bloom_level }}</span></td>
      <td>{{ clo.program_outcomes_mapped | join(', ') }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>

<!-- Section III: 18-Week Detailed Schedule with Tripartite Domains -->
<div class="section-title">III. 18-Week Detailed Teaching-Learning Schedule</div>
<table>
  <thead>
    <tr>
      <th>Week</th><th>Topics / Subject Matter</th><th>Lesson Outcomes (K/S/A)</th>
      <th>TLAs</th><th>Assessment Tasks</th>
    </tr>
  </thead>
  <tbody>
    {% for w in schedule %}
    <tr class="{% if w.week_number in [9, 18] %}exam-row{% endif %}">
      <td><strong>Week {{ w.week_number }}</strong></td>
      <td>{{ w.topics | join('; ') }}</td>
      <td>
        {% for llo in w.lesson_outcomes %}
        <div><span class="badge badge-{{ llo.domain | lower }}">{{ llo.domain }}</span> {{ llo.descript

```



```
ion }}</div>
    {% endfor %}
  </td>
  <td>{{ w.teaching_learning_activities | join('; ') }}</td>
  <td>{{ w.assessment_tasks | join('; ') }}</td>
</tr>
{% endfor %}
</tbody>
</table>

<!-- Institutional Signatories -->
<div class="sign-grid">
  <div class="sign-box"><div class="sign-line">Faculty Member</div><div class="sign-title">Prepared By
</div></div>
  <div class="sign-box"><div class="sign-line">Department Chair, CS</div><div class="sign-title">Verifi
ed & Recommended</div></div>
  <div class="sign-box"><div class="sign-line">Dean, CCS</div><div class="sign-title">Approved By</div>
</div>
</div>
```

3.4 Deliverable 4: SQLite Query & HTML/PDF Syllabus Compilation (export_engine.py)

Queries the SQLite relational database by course_code and renders an official, browser-ready HTML/PDF syllabus document using Jinja2:

```
from pathlib import Path
from typing import Optional
from jinja2 import Environment, FileSystemLoader, select_autoescape
from db_manager import get_syllabus, DEFAULT_DB_PATH
from obe_schemas import FullSyllabusSchema

BASE_DIR = Path(__file__).resolve().parent
TEMPLATES_DIR = BASE_DIR / "templates"
TEMPLATE_FILENAME = "uphsd_ccs_template.html"

def get_jinja_env() -> Environment:
    '''Configures and returns the Jinja2 template environment with autoescaping.'''
    if not TEMPLATES_DIR.exists():
        raise FileNotFoundError(f"Templates directory not found at '{TEMPLATES_DIR}'.")
    return Environment(
        loader=FileSystemLoader(str(TEMPLATES_DIR)),
        autoescape=select_autoescape(["html", "xml"])
    )

def render_syllabus_html(course_code: str, db_path: str = DEFAULT_DB_PATH) -> str:
    '''Queries SQLite for course_code and compiles the official UPHSD CCS syllabus HTML.'''
    syllabus: Optional[FullSyllabusSchema] = get_syllabus(course_code, db_path=db_path)
    if not syllabus:
        raise ValueError(f"No syllabus records found in database for course code '{course_code}'.")

    env = get_jinja_env()
    template = env.get_template(TEMPLATE_FILENAME)
    return template.render(
        course=syllabus.course_metadata,
        clos=syllabus.course_outcomes,
        schedule=syllabus.weekly_schedule
    )

def export_to_file(course_code: str, output_path: Optional[str] = None, db_path: str = DEFAULT_DB_PATH) -> str:
    '''Compiles HTML syllabus and saves it to a persistent output deliverable file.'''
    html_content = render_syllabus_html(course_code, db_path=db_path)
    if not output_path:
        clean_code = course_code.replace(" ", "_").replace("/", "-")
        outputs_dir = BASE_DIR / "outputs"
        outputs_dir.mkdir(parents=True, exist_ok=True)
        output_file = outputs_dir / f"official_syllabus_{clean_code}.html"
    else:
        output_file = Path(output_path)

    output_file.parent.mkdir(parents=True, exist_ok=True)
    with open(output_file, "w", encoding="utf-8") as f:
        f.write(html_content)

    print(f"[+] Successfully exported official syllabus to '{output_file}'.")
    return str(output_file.resolve())
```

3.5 Relational Persistence & Human-in-the-Loop CRUD (`db_manager.py`)

Provides transactional persistence, atomic parent-child cascade deletions, and faculty CLO editing:

```

def save_syllabus(syllabus: FullSyllabusSchema, db_path: str = DEFAULT_DB_PATH) -> int:
    '''Atomically persists full syllabus across all 4 relational tables.'''
    with get_connection(db_path) as conn:
        with conn:
            # 1. Insert course metadata
            cur = conn.execute(
                "INSERT OR REPLACE INTO courses (course_code, course_title, course_description, "
                "credit_units, lecture_hours, lab_hours, prerequisites) VALUES (?, ?, ?, ?, ?, ?, ?)",
                (m.course_code, m.course_title, m.course_description, m.credit_units,
                 m.lecture_hours, m.lab_hours, m.prerequisites)
            )
            course_id = cur.lastrowid

            # 2. Insert Course Learning Outcomes
            for clo in syllabus.course_outcomes:
                conn.execute(
                    "INSERT INTO course_outcomes (course_id, clo_id, description, bloom_level, "
                    "program_outcomes_mapped) VALUES (?, ?, ?, ?, ?)",
                    (course_id, clo.clo_id, clo.description, clo.bloom_level,
                     json.dumps(clo.program_outcomes_mapped))
                )

            # 3. Insert Weekly Schedules & Lesson Outcomes
            for week in syllabus.weekly_schedule:
                w_cur = conn.execute(
                    "INSERT INTO weekly_schedules (course_id, week_number, topics, "
                    "teaching_learning_activities, assessment_tasks) VALUES (?, ?, ?, ?, ?)",
                    (course_id, week.week_number, json.dumps(week.topics),
                     json.dumps(week.teaching_learning_activities), json.dumps(week.assessment_tasks))
                )
                sched_id = w_cur.lastrowid
                for llo in week.lesson_outcomes:
                    conn.execute(
                        "INSERT INTO lesson_outcomes (schedule_id, llo_id, domain, description) "
                        "VALUES (?, ?, ?, ?)",
                        (sched_id, llo.llo_id, llo.domain, llo.description)
                    )
            return course_id

def update_clo(clo_id: str, new_description: str, db_path: str = DEFAULT_DB_PATH) -> bool:
    '''Updates a CLO statement after validating active Bloom verbs.'''
    CourseOutcomeSchema.validate_action_verbs(new_description)
    with get_connection(db_path) as conn:
        with conn:
            conn.execute("UPDATE course_outcomes SET description = ? WHERE clo_id = ?",
                          (new_description, clo_id))
    return True

def delete_course(course_code: str, db_path: str = DEFAULT_DB_PATH) -> bool:
    '''Nuclear deletes a specific course and all its cascaded relational records.'''
    norm_code = course_code.strip()
    with get_connection(db_path) as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT id FROM courses WHERE UPPER(TRIM(course_code)) = UPPER(?);", (norm_cod
e,))
        rows = cursor.fetchall()
        for row in rows:
            cid = row["id"]
            cursor.execute("DELETE FROM lesson_outcomes WHERE schedule_id IN (SELECT id FROM weekly_sch

```

```

        edules WHERE course_id = ?);", (cid,))
        cursor.execute("DELETE FROM weekly_schedules WHERE course_id = ?;", (cid,))
        cursor.execute("DELETE FROM course_outcomes WHERE course_id = ?;", (cid,))
        cursor.execute("DELETE FROM courses WHERE id = ?;", (cid,))
        conn.commit()
        conn.execute("VACUUM;")
    return True

```

3.6 Selective Synthesis & Batch Worker (app.py)

Enables selective batch generation that skips existing courses to save computation time:

```

def run_batch_generation_worker(skip_existing: bool = False):
    all_subjects = get_all_subjects()
    if skip_existing:
        queue = [s for s in all_subjects if not is_subject_generated(s["course_code"])]
        skipped = [s for s in all_subjects if is_subject_generated(s["course_code"])]
    else:
        queue = all_subjects
        skipped = []

    total = len(queue)
    for idx, subj in enumerate(queue, 1):
        code = subj["course_code"]
        # Progress callback calculating granular percentage
        base_pct = int(((idx - 1) / total) * 100)
        subj_weight = 100.0 / total
        llm_engine.generate_custom_subject(course_code=code, progress_callback=make_batch_progress())

```